深度研究报告

bilibili-api-python 在 video-note-pipeline 中的集成研究

面向 Bilibili 官方字幕 fallback 的实现、依赖、交互与持久化设计

Codex

2026-03-26

本报告未使用封面图，重点保留可审计的源码、文档与本地实验 artifact。

核心问题：在保留现有 `yt-dlp` 主链路的前提下，`bilibili-api-python` 是否适合接入 `video-note-pipeline` 作为 Bilibili 专用 fallback，以及它应该如何安装、认证、调用、落盘与暴露给用户。

研究范围：聚焦当前本机仓库 `/home/fanghaotian/Desktop/src/video-note-pipeline` 的 Bilibili 字幕探测与媒体探测流程；不覆盖 YouTube，不设计最终 change，只给出可执行的工程结论。

目标读者：准备为 `video-note-pipeline` 编写下一轮 change / propose / implementation 的开发者。

证据窗口：官方资料与本地实验访问日期均截至 2026-03-26（Asia/Shanghai）。

目录

1 执行摘要 **3**

2 研究范围与方法 **4**
2.1 证据清单 4

3 当前项目上下文 **6**

4 官方能力与安装模型 **6**
4.1 安装与依赖结构 6
4.2 请求客户端与网络设置 7
4.3 许可证与分发问题 7

5 认证模型与用户交互 **8**
5.1 Credential 需要什么 8
5.2 当前项目 cookies 能否直接复用 8
5.3 是否要暴露 QR 登录 / 密码登录 / 短信登录 9

6 它能拿到什么信息 **9**
6.1 对字幕链路最关键的接口 9
6.2 公开访问与认证访问的实际边界 10
6.3 字幕接口真正返回什么 10

7 本地实验结果 **11**
7.1 实验环境 11
7.2 关键实验一览 12
7.3 目标视频的最重要证据 12

8 应该如何接到当前项目里 **13**
8.1 推荐架构 13
8.2 依赖接入建议 14
8.3 同步/异步边界建议 14
8.4 推荐的落盘策略 15
8.5 字段归一化建议 15
8.6 多 P 与 URL 语义 15

9 哪些地方最容易踩坑 **16**

10 方案比较与推荐决策 **17**

11 结论与后续问题 **17**
11.1 已经有充分支持的结论 17
11.2 很可能成立，但仍建议在实现时再验证的判断 18
11.3 仍未知或值得单独验证的问题 18

A 复现实验命令 **18**

B 来源清单 **19**

1 执行摘要

核心结论

`bilibili-api-python` 适合接入当前项目，但最佳位置不是替换 `yt-dlp`，而是作为 **Bilibili 专用、带 cookies 的第二阶段 fallback**。理由有三点：

- 它能稳定拿到当前 `yt-dlp` 路径之外的 Bilibili 原生字幕元信息，关键接口是 `Video.get_subtitle()` / `Video.get_player_info()`，并且本地已实测拿到 `ai-zh` 字幕。
- 它可以直接复用我们现有的浏览器导出 cookies 文件，不需要引入新的登录交互；通过 `Credential.from_cookies(...)` 即可桥接。
- 它是全异步、依赖更重、且仓库许可证为 GPLv3+，因此作为全局主链路替代方案的代价偏高；放在 Bilibili fallback 层，收益和复杂度最平衡。

推荐工程策略

面向下一轮代码实现，建议采用以下策略：

- 保留当前 `yt-dlp` Bilibili 字幕路径作为第一层。
- 当 `yt-dlp` 在 cookies 路径下仍未选中官方字幕时，且本地已有有效 Bilibili cookies，则进入 `bilibili-api-python` fallback。
- fallback 内部复用已有 cookie 文件，调用 `get_pages()`、`get_cid()`、`get_subtitle()`，然后对返回的 `subtitle_url` 再发一次 HTTP GET，落盘原始 JSON，并归一化成当前项目已有的 transcript payload。
- 用户侧不新增 QR 登录、短信登录、密码登录命令，继续统一使用 `video-note cookies-export bilibili`。

本轮调研确认的主要风险

- **许可证风险**：官方 README 明示 GPLv3+。如果 `video-note-pipeline` 会以依赖形式分发，必须先做许可证评估。
- **刷新风险**：`credential.refresh()` 需要 `ac_time_value`。我们当前导出的 Netscape cookies 文件没有这个字段，因此不能直接依赖库内刷新逻辑。
- **异步边界**：该库“全部是异步操作”。当前 CLI 以同步风格组织，因此需要一个边界层来管理事件循环与超时。
- **原始字幕不是最终 transcript**：`get_subtitle()` 返回的是字幕元信息，不是项目可直接消费的标准 transcript，仍需二次请求与归一化。

2 研究范围与方法

研究方法

本报告没有只读文档就下结论，而是把官方资料与本地可运行实验绑定在一起：

- 在线核验官方 README、PyPI、官方文档入口，确认版本、安装方式、请求客户端和登录模型。
- 在隔离环境内真实安装 `bilibili-api-python==17.4.1`，并额外安装 `httpx` 与 `curl_cffi` 做可运行验证。
- 直接复用当前机器上的 `/home/fanghaotian/.config/video-note-pipeline/cookies/bilibili.txt`，验证能否转成 `Credential` 并拿到目标视频的字幕。
- 对照当前仓库里的 Bilibili adapter 与 subtitle probe 实现，分析应该如何接入、如何落盘、如何保持用户交互稳定。

2.1 证据清单

ID	模态	标题 / 标签	链接或路径
S1	Web	官方 README	https://github.com/Nemo2011/bilibili-api
S2	Web	官方文档入口	https://nemo2011.github.io/bilibili-api/
S3	Web	PyPI 项目页	https://pypi.org/project/bilibili-api-python/
S4	Web/Repo	获取 Credential 文档	https://github.com/Nemo2011/bilibili-api/blob/main/docs/get-credential.md
S5	Web/Repo	刷新 cookies 文档	https://github.com/Nemo2011/bilibili-api/blob/main/docs/refresh_cookies.md
S6	Web/Repo	请求客户端文档	https://github.com/Nemo2011/bilibili-api/blob/main/docs/request_client.md
S7	Web/Repo	配置文档	https://github.com/Nemo2011/bilibili-api/blob/main/docs/configuration.md
S8	Web/Repo	Video 模块文档	https://github.com/Nemo2011/bilibili-api/blob/main/docs/modules/video.md
S9	Web/Repo	Login v2 模块文档	https://github.com/Nemo2011/bilibili-api/blob/main/docs/modules/login_v2.md

ID	模态	标题 / 标签	链接或路径
S10	Local code	当前项目 Bilibili adapter	/home/fanghaotian/Desktop/src/video-note-pipeline/src/video_note_pipeline/adapters/bilibili.py
S11	Local code	当前 subtitle probe 组装逻辑	/home/fanghaotian/Desktop/src/video-note-pipeline/src/video_note_pipeline/probes/subtitles.py
S12	Local code	当前项目 pyproject 依赖声明	/home/fanghaotian/Desktop/src/video-note-pipeline/pyproject.toml
S13	Local experiment	cookies 转 Credential 验证	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/credential-check.json
S14	Local experiment	refresh 失败原因验证	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/credential-refresh-attempt.json
S15	Local experiment	public vs auth 行为差异	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/public-vs-auth-behavior.json
S16	Local experiment	字幕元信息与原始 JSON 摘要	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/subtitle-metadata-and-json-summary.json
S17	Local experiment	原始字幕 JSON 样本	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/subtitle-json-sample.json
S18	Local experiment	下载地址能力摘要	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/download-url-summary.json
S19	Local experiment	QR 登录能力摘要	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/qrcode-login-summary.json

ID	模态	标题 / 标签	链接或路径
S20	Local experiment	curl_cffi 实测字幕检查	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/curl-cffi-subtitle-check.json

3 当前项目上下文

当前项目的 Bilibili 逻辑仍然是 `yt-dlp` 主导。以 2026-03-26 本地代码为准，Bilibili adapter 的几个关键特征如下：

- 字幕请求走 `yt-dlp`，并且语言请求现在已经扩大为 `all,-live_chat,-danmaku`。
- 语言归一化已经能识别 `ai-zh`、`auto-zh`、`zh-Hans`、`zh-Hant` 等标签，并将其落入统一选择逻辑。
- subtitle probe 的输出已经保留 `raw_tracks`、`normalized_tracks`、`excluded_tracks` 与 `selection_trace`，具备较好的可观测性。
- cookies 获取已通过 `video-note cookies-export bilibili` 固化，并以 Netscape cookie file 形式存放在用户配置目录。

为什么还要调研 bilibili-api-python

虽然前一轮已经修复了 Bilibili `yt-dlp` 对 `ai-zh` 的识别问题，但 `yt-dlp` 本质上仍是站外抽象层。它能拿到 sidecar 字幕时很好用，但它并不暴露 Bilibili 原生的 `player_info`、`need_login_subtitle`、`subtitle_url` 等站内字段。对于“cookies 有效但 `yt-dlp` 仍未返回可用字幕”的 case，Bilibili 专用 fallback 仍然有价值。

4 官方能力与安装模型

4.1 安装与依赖结构

官方 README 明确给出安装方式：

- `pip install bilibili-api-python`
- 另行安装一个异步请求库：`aiohttp`、`httpx` 或 `curl_cffi`

本地隔离环境的实测安装版本是 `bilibili-api-python==17.4.1`。直接元数据依赖如下：

- `beautifulsoup4`
- `colorama`
- `lxml`
- `pyyaml`
- `brotli`

- qrcode
- APScheduler
- pillow
- yarl
- pycryptodomex
- qrcode_terminal
- 以及单独选择的 HTTP 客户端，例如 httpx 或 curl_cffi

方案	需要安装	适合当前项目的原因
最小接入	bilibili-api-python + httpx	本地已验证能拿到目标视频的 ai-zh 字幕，依赖较轻，足够作为 fallback 起步。
增强抗风控	bilibili-api-python + curl_cffi	官方优先级更高，支持 impersonation。若后续遇到 412 或风控波动，可启用。
不推荐现在就做	直接替换现有 yt-dlp Bilibili 主链路	需要引入更多异步边界、许可证评估和额外归一化代码，当前收益不如 fallback 明确。

表 2: 面向当前项目的安装策略

4.2 请求客户端与网络设置

官方文档明确指出默认支持三类异步请求库，并按 curl_cffi > aiohttp > httpx 的顺序选择。文档还给出如下可配置项：

- 通用：proxy、timeout、verify_ssl、trust_env
- 库特有：impersonate、http2
- Bilibili 特有：wbi_retry_times、enable_auto_buvid、enable_bili_ticket

对当前项目的建议

第一阶段默认用 httpx。原因是：

- 我们已经在隔离环境里用 httpx 成功完成了 Bilibili 字幕元信息与字幕 JSON 的获取。
- 它比 curl_cffi 更容易维护，且 video-note-pipeline 当前不依赖 WebSocket。
- 若后续遇到风控问题，再引入可选增强模式：select_client("curl_cffi") + request_settings.set("impersonate", "...") 。

4.3 许可证与分发问题

官方 README 明确声明项目遵循 **GPLv3+**。这不是实现细节，而是设计边界：

- 如果 `video-note-pipeline` 只是本机工具或内部分发，依赖引入的工程阻力较小，但仍应在变更文档中显式记录。
- 如果它将来要以闭源或混合许可方式对外分发，必须在引入前完成许可证评估。

5 认证模型与用户交互

5.1 Credential 需要什么

官方文档和源码都表明，`Credential` 可以从以下字段构造：

- `SESSDATA`
- `bili_jct`
- `buvid3`
- `buvid4`
- `DedeUserID`
- `ac_time_value`

其中 `ac_time_value` 只在 cookies 刷新时需要，来自浏览器 `localStorage`，并不在普通 cookie file 里。

5.2 当前项目 cookies 能否直接复用

答案是能，而且已经本地实测成功。当前 `video-note-pipeline` 导出的 Bilibili cookies 文件中，以下关键信息都能取到：

- `SESSDATA`
- `bili_jct`
- `buvid3`
- `buvid4`
- `DedeUserID`

对应的本地 artifact 显示：

- `check_valid = true`
- `check_refresh = true`
- 但 `ac_time_value` 缺失

关于 refresh 的实际结论

本地已经直接验证：当前导出的 cookie file **不能**直接调用 `credential.refresh()`。错误是 `CredentialNoAcTimeValueException`。因此，若接入本库，推荐的策略不是“自动 refresh cookies”，而是继续沿用现有用户动作：

- cookie 失效时重新执行 `video-note cookies-export bilibili`
- 不新增“自动刷新 Bilibili cookies”命令

5.3 是否要暴露 QR 登录 / 密码登录 / 短信登录

官方库支持：

- `QrCodeLogin`
- 密码登录
- 短信登录

本地也已经验证 `QrCodeLogin` 的二维码生成路径存在。但对当前项目而言，这不是推荐方向：

- 我们已经有更稳定的浏览器 cookie 导出链路。
- 新增登录交互会增加终端状态机、超时轮询、二维码展示与失败恢复复杂度。
- 从用户心智上，统一“先在浏览器登录，再导出 cookies”更简单。

用户交互建议

如果未来接入 `bilibili-api-python`，用户不应感知到新命令。推荐交互如下：

- 正常用户：继续只用 `video-note cookies-export bilibili --browser edge --json`
- pipeline：自动检测已有 cookie file，并在 Bilibili fallback 中复用
- 失败时：继续提示重导 cookies，而不是要求用户扫描二维码或粘贴 token

6 它能拿到什么信息

6.1 对字幕链路最关键的接口

本次调研里最有价值的是以下几个视频接口：

- `Video.get_info()`：公开视频基础信息
- `Video.get_pages()`：获取分 P 信息
- `Video.get_cid()`：定位具体页的 `cid`
- `Video.get_player_info(cid=...)`：返回字幕、地区与最近播放信息，含字幕 JSON 链接
- `Video.get_subtitle(cid=...)`：从 `player_info` 中抽出字幕区块
- `Video.get_download_url(...)`：返回播放/下载地址信息

6.2 公开访问与认证访问的实际边界

对目标视频 BV1DzALzQEHU 的本地实验结果非常清楚：

- get_info()、get_pages()、get_cid() 在匿名状态下可用。
- 匿名调用 get_player_info() 会报 CredentialNoSessdataException。
- 带当前 cookie file 转换得到的 Credential 后，get_subtitle() 返回 1 条字幕轨，语言为 ai-zh。

能力	匿名访问	带当前 cookie file 的访问
get_info() / get_pages() / get_cid()	可用	可用
get_player_info() / get_subtitle()	不可用，缺 SESSDATA	可用，返回 ai-zh 字幕轨
get_download_url()	可用，但 DASH 视频流更少	可用，且本地实测看到更多 DASH 视频流

表 3: Bilibili 目标视频的公开/认证能力对比

6.3 字幕接口真正返回什么

get_subtitle() 并不直接返回标准化后的 transcript，而是返回一个“字幕索引对象”。实测字段包括：

- subtitles[].lan，例如 ai-zh
- subtitles[].lan_doc，例如“中文”
- subtitles[].subtitle_url
- subtitles[].subtitle_url_v2
- subtitles[].ai_type / ai_status

其中 subtitle_url 是协议相对地址，例如 //aisubtitle...。代码接入时必须先规范化为 https://... 再发起 HTTP GET。

拿到实际 JSON 后，顶层包含：

- font_size
- font_color
- background_alpha
- background_color
- Stroke
- type
- lang

- version
- body[]

  其中真正要转成 transcript 的是 body[]：

- from：起始秒数
- to：结束秒数
- content：字幕文本
- 其他补充字段如 sid、location、music

```python
import asyncio
from pathlib import Path
from bilibili_api import Credential, video, select_client


def load_netscape_cookies(cookie_file: Path) -> dict[str, str]:
    cookies = {}
    for raw in cookie_file.read_text(encoding="utf-8").splitlines():
        if not raw or raw.startswith("#"):
            continue
        parts = raw.split("\t")
        if len(parts) >= 7:
            cookies[parts[5]] = parts[6]
    return cookies


async def fetch_subtitle_index(bvid: str, cookie_file: Path) -> dict:
    select_client("httpx")
    credential = Credential.from_cookies(load_netscape_cookies(cookie_file))
    v = video.Video(bvid=bvid, credential=credential)
    cid = await v.get_cid(0)
    return await v.get_subtitle(cid=cid)


subtitle_index = asyncio.run(
    fetch_subtitle_index(
        "BV1DzALzQEHU",
        Path("/home/fanghaotian/.config/video-note-pipeline/cookies/bilibili.txt"),
    )
)
```

Listing 1: 将现有 cookie file 转为 Credential 并获取字幕元信息的最小原型

7 本地实验结果

7.1 实验环境

- 操作系统：Arch Linux（当前机器）

- 项目路径：`/home/fanghaotian/Desktop/src/video-note-pipeline`
- 报告工作目录：`/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration`
- 安装版本：`bilibili-api-python==17.4.1`
- 已验证请求客户端：`httpx`、`curl_cffi`

7.2 关键实验一览

实验	结果	工程含义
cookie file 转 Credential	成功	现有 `cookies-export` 结果可以直接复用，不需要新登录流程。
Credential 有效性检查	`check_valid = true`	当前浏览器导出的正式 cookie 文件可以作为 fallback 输入。
Credential 刷新检查	`check_refresh = true` 但 `refresh()` 失败	缺 `ac_time_value`，不要依赖库内 refresh。
匿名字幕探测	失败	仅靠匿名 API 不足以获得 Bilibili 官方字幕。
带 cookies 的字幕探测	成功得到 `ai-zh`	能补足当前 `yt-dlp` 可能漏掉的官方字幕路径。
原始字幕 JSON 下载	成功，`body_len = 387`	fallback 不止能知道“有字幕”，还足以生成标准 transcript。
下载地址探测	公开和认证都可用，认证流更丰富	未来可作为视频画质探测的补充信息来源，但不建议本轮替换下载栈。
`curl_cffi` 实测	成功	可作为遇到风控时的增强模式。
QR 登录验证	能生成二维码，未完成整链路登录	功能存在，但不建议暴露到当前产品交互。

表 4: 本地实验摘要

7.3 目标视频的最重要证据

对用户关心的目标视频 `BV1DzALzQEHU`，本地已经拿到了以下关键结果：

- `public-vs-auth-behavior.json` 显示：公开可读 `title`、`pages_count` 和 `cid`，但公开 `player_info` 会因为缺 `SESSDATA` 失败。
- 同一份 artifact 也显示：带认证后 `auth_subtitle_count = 1`，且第一条 `lan = ai-zh`。
- `subtitle-metadata-and-json-summary.json` 显示：字幕索引对象中存在 `subtitle_url` 与 `subtitle_url_v2`。

- 原始字幕 JSON 样本大小约 69 KB，包含 387 段 `body[]`，足够归一化为项目 transcript。

一个非常具体的判断

这次实验说明，`bilibili-api-python` 的价值不在“猜语言标签”，而在于它能直接拿到 Bilibili 站内字幕索引和原始字幕 JSON 链接。也就是说，它最适合兜底“`yt-dlp` cookies 路径仍 miss，但 B 站官方字幕实际存在”的场景。

8 应该如何接到当前项目里

8.1 推荐架构

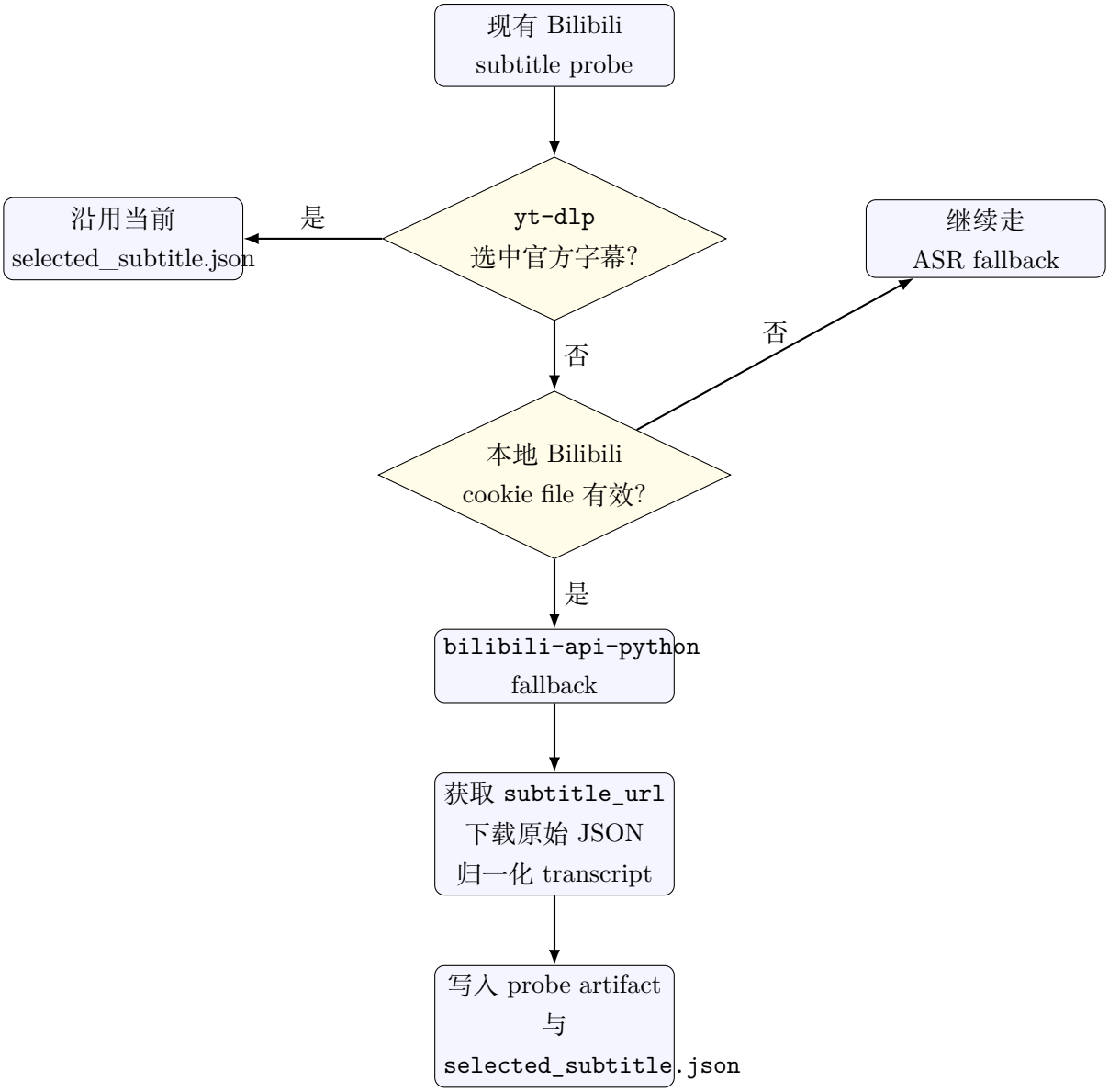

图 1: 推荐的 Bilibili 字幕 fallback 架构

8.2 依赖接入建议

如果后续要写代码，建议不要直接把依赖塞进基础依赖组，而是新增一个 Bilibili 专用可选组。例如：

```
[project.optional-dependencies]
bilibili-direct = [
  "bilibili-api-python>=17.4.1",
  "httpx>=0.28.0",
]
bilibili-direct-hardened = [
  "bilibili-api-python>=17.4.1",
  "curl_cffi>=0.14.0",
]
```

Listing 2: 建议的 pyproject optional dependency 方向

这样做的优点：

- 不影响 YouTube 用户或不需要 Bilibili fallback 的用户。
- 把 GPLv3+ 依赖的进入点缩小到明确的 optional feature。
- 让 `httpx` 与 `curl_cffi` 的选择更清晰。

8.3 同步/异步边界建议

当前项目是同步 CLI，`bilibili-api-python` 是全异步库。建议新增一个很薄的内部模块，例如 `video_note_pipeline/adapters/bilibili_api_fallback.py`：

- 内部写一个 `async def _probe_subtitle_index(...)` 异步函数。
- 外层暴露同步函数，由 `asyncio.run(...)` 驱动。
- 不要把异步对象泄漏到现有 command / adapter 边界之外。

官方文档也提供 `sync()` 包装器，但从项目可读性和边界控制上，显式的内部 async wrapper 更容易审计。

8.4 推荐的落盘策略

文件名建议	内容	为什么要存
`bilibili_api_probe.json`	本次 fallback 的选择客户端、cid、错误、是否命中字幕	让 probe 失败原因可调试，不依赖运行日志。
`bilibili_player_info.json`	`get_player_info()` 原始返回	记录 `need_login_subtitle`、`subtitle`、地区等站内信息。
`bilibili_subtitle_index.json`	`get_subtitle()` 返回的字幕索引对象	记录每条字幕轨的 `lan`、`lan_doc`、`subtitle_url`。
`bilibili_subtitle_raw.<lan>.json`	通过 `subtitle_url` 下载的原始字幕 JSON	保留源数据，便于后续归一化逻辑调整时重放。
`selected_subtitle.json`	归一化后的最终字幕 payload	与现有 pipeline 约定保持一致，让后续写作 / ASR fallback 无感知。

表 5: 推荐的 artifact 保存策略

8.5 字段归一化建议

从当前样本看，最低可行归一化规则如下：

- `source`：建议写成 `official-cookies:bilibili-api-python`
- `language`：保留原始 `lan`，例如 `ai-zh`
- `normalized_language`：映射到现有项目惯例，例如 `zh`
- `segments[]`：由 `body[]` 映射，每段至少保留 `start`、`end`、`text`
- `provenance`：补充 `subtitle_url`、`cid`、`selected_client`

为什么原始 JSON 也要存

因为 Bilibili 返回的不只是文本段，还有 `type`、`lang`、`music`、`location` 等辅助字段。今天我们可能只取 `from/to/content`，但后续若想做歌词过滤、音乐置信度过滤、字幕位置分析，必须保留原始数据。

8.6 多 P 与 URL 语义

官方文档说明 `page_index` 与 `cid` 至少提供一个，且 `cid` 优先。对当前项目，建议：

- 先沿用现有 metadata 获取逻辑决定目标页。
- 若 URL 已带 `p=`，则优先把它解析成对应的 `cid`。
- fallback 只消费“已经确定好的页”，不要在 Bilibili 专用层重新发明页面选择逻辑。

9 哪些地方最容易踩坑

坑 1：把它当作“直接拿 transcript 的库”

不是。它直接给我们的，是 Bilibili 原生字幕索引和原始 JSON 链接，而不是已经符合项目 schema 的 transcript。实现时必须再做一次 HTTP GET 和归一化。

坑 2：以为现有 cookie file 可以自动 refresh

不是。当前 cookies 文件能登录、能看字幕，但没有 `ac_time_value`。如果实现里误调用 `credential.refresh()`，会直接失败。

坑 3：把登录能力也一起引进来

技术上可以，产品上不应该。QR 登录、短信登录、密码登录会让当前项目多出一整套交互、状态轮询和错误恢复路径，明显超出这一轮目标。

坑 4：忽略许可证

官方仓库明确是 GPLv3+。这类问题不能等代码写完再看。

坑 5：在主链路里过早替换 yt-dlp

`yt-dlp` 仍然承担媒体下载、跨平台 cookie 导出与部分字幕能力；完全替换会把风险面扩大。当前最合理的方式是把 `bilibili-api-python` 放进 Bilibili 专用 fallback 层。

10 方案比较与推荐决策

方案	官方字幕命中能力	依赖/复杂度	对用户交互影响	结论
只保留 `yt-dlp`	中等，依赖 sidecar 结果	最低	无新增交互	已是基础能力，但无法覆盖所有 Bilibili 站内字幕场景。
增加 `bilibili-api-python` fallback	高，能直接读站内字幕索引与原始 JSON	中等	无新增交互，可复用 cookie export	**推荐**。收益最大，工程代价可控。
全面改为 `bilibili-api-python` 主链路	高，但需要补下载、事件循环、更多观测与许可证评估	最高	可能引入新调试与登录问题	现在不推荐。
引入 `bilibili-api-python` 并开放 QR 登录	技术可行	很高	新增交互明显	不符合当前项目产品方向。

表 6: Bilibili 字幕方案比较

最终建议

如果下一步是写 change / propose，我建议把目标描述成：

- “为 `video-note-pipeline` 增加 Bilibili 专用 `bilibili-api-python` 字幕 fallback，复用现有 cookie export，不新增用户登录交互，并保留原始字幕 JSON artifact。”

而不要写成：

- “把 Bilibili 全面迁移到 `bilibili-api-python`”，
- “增加二维码登录”，
- “做自动 cookies refresh”。

11 结论与后续问题

11.1 已经有充分支持的结论

- `bilibili-api-python` 可以直接复用当前项目导出的 Bilibili cookies 文件。
- 对目标视频 `BV1DzALzQEHU`，它在本机上已经成功取回 `ai-zh` 字幕索引与原始字幕 JSON。
- 它适合放在 Bilibili fallback 层，而不是替换现有 `yt-dlp` 主链路。
- 接入后不需要新增用户登录交互，继续使用 `video-note cookies-export bilibili` 即可。
- 真正可用的持久化方案必须同时保存原始站内 JSON 和项目归一化 transcript。

11.2 很可能成立，但仍建议在实现时再验证的判断

- `httpx` 作为第一阶段默认客户端已经足够；只有在遇到 412 或更强风控时才需要切到 `curl_cffi`。
- `get_download_url()` 未来可能帮助视频画质探测，但这一轮没有必要动当前下载栈。
- 当前 Bilibili fallback 的最佳触发条件，是 `yt-dlp` cookies 路径 miss 后再进入，而不是并行双探测。

11.3 仍未知或值得单独验证的问题

- 许可证约束在该项目的分发模式下是否可接受。
- 多 P 视频、合集、番剧、课程等变体在 fallback 层是否需要单独处理。
- 在更严格风控网络环境下，`httpx` 是否仍然足够稳定，还是需要默认 `curl_cffi`。
- `subtitle_url_v2` 是否比 `subtitle_url` 更稳定，是否值得优先使用。

A 复现实验命令

```
cd /home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-
    python-integration
uv venv artifacts/bapi-venv
uv pip install --python artifacts/bapi-venv/bin/python bilibili-api-python httpx
uv pip install --python artifacts/bapi-venv/bin/python curl_cffi
```

Listing 3: 隔离环境中安装并验证 bilibili-api-python 的参考命令

```
source artifacts/bapi-venv/bin/activate
python - <<'PY'
import asyncio
from pathlib import Path
from bilibili_api import Credential, video, select_client

def load_netscape_cookies(cookie_file: Path) -> dict[str, str]:
    cookies = {}
    for raw in cookie_file.read_text(encoding="utf-8").splitlines():
        if not raw or raw.startswith("#"):
            continue
        parts = raw.split("\t")
        if len(parts) >= 7:
            cookies[parts[5]] = parts[6]
    return cookies

async def main() -> None:
    select_client("httpx")
    credential = Credential.from_cookies(
        load_netscape_cookies(
            Path("/home/fanghaotian/.config/video-note-pipeline/cookies/bilibili.txt")
        )
    )
```

```
    v = video.Video(bvid="BV1DzALzQEHU", credential=credential)
    cid = await v.get_cid(0)
    subtitle = await v.get_subtitle(cid=cid)
    print(subtitle["subtitles"][0]["lan"])

asyncio.run(main())
PY
```

Listing 4: 验证当前 cookie file 能否拿到目标视频字幕的参考命令

B 来源清单

ID	类型	来源	链接 / 路径
S1	Primary	bilibili-api 官方 README	https://github.com/Nemo2011/bilibili-api
S2	Primary	bilibili-api 官方文档入口	https://nemo2011.github.io/bilibili-api/
S3	Primary	PyPI 项目页	https://pypi.org/project/bilibili-api-python/
S4	Primary	获取 Credential 文档	https://github.com/Nemo2011/bilibili-api/blob/main/docs/get-credential.md
S5	Primary	refresh cookies 文档	https://github.com/Nemo2011/bilibili-api/blob/main/docs/refresh_cookies.md
S6	Primary	请求客户端文档	https://github.com/Nemo2011/bilibili-api/blob/main/docs/request_client.md
S7	Primary	配置文档	https://github.com/Nemo2011/bilibili-api/blob/main/docs/configuration.md
S8	Primary	Video 模块文档	https://github.com/Nemo2011/bilibili-api/blob/main/docs/modules/video.md
S9	Primary	Login v2 模块文档	https://github.com/Nemo2011/bilibili-api/blob/main/docs/modules/login_v2.md
S10	Repo	当前项目 Bilibili adapter	/home/fanghaotian/Desktop/src/video-note-pipeline/src/video_note_pipeline/adapters/bilibili.py
S11	Repo	当前 subtitle probe 逻辑	/home/fanghaotian/Desktop/src/video-note-pipeline/src/video_note_pipeline/probes/subtitles.py
S12	Repo	当前依赖声明	/home/fanghaotian/Desktop/src/video-note-pipeline/pyproject.toml
S13	Experiment	Credential 可用性	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/credential-check.json
S14	Experiment	refresh 失败摘要	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/credential-refresh-attempt.json

ID	类型	来源	链接 / 路径
S15	Experiment	公开/认证差异	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/public-vs-auth-behavior.json
S16	Experiment	字幕元信息与 JSON 摘要	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/subtitle-metadata-and-json-summary.json
S17	Experiment	原始字幕 JSON	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/subtitle-json-sample.json
S18	Experiment	下载地址能力摘要	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/download-url-summary.json
S19	Experiment	QR 登录能力摘要	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/qrcode-login-summary.json
S20	Experiment	`curl_cffi` 字幕检查	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-api-python-integration/artifacts/curl-cffi-subtitle-check.json